題目：員工證號自動生成系統 (Employee ID System)

- **核心概念**：Class Attribute (類別屬性) vs Instance Attribute (實體屬性)
- **情境描述**： 你正在為一家新創公司開發人資系統。每當有一位新員工入職，我們需要建立一個員工檔案（物件）。 這家公司的員工編號是**自動遞增**的，第一位員工編號是 1，第二位是 2，依此類推。 此外，公司隨時想知道「目前總共有多少位員工」。
    - **為什麼要用類別屬性？** 因為「下一個可用的 ID」和「總員工數」是全公司共享的資訊，不屬於某個特定員工。
- **程式要求**： 設計 Employee 類別。
    1. 類別屬性 total_employees (初始為 0) 和 next_id (初始為 1)。
    2. 建構子 __init__(self, name)：設定員工名字，自動分配 id，並更新類別屬性。
- **輸入格式**： 第一行整數 N。接下來 N 行，每行一個員工姓名。
- **輸出格式**： 每建立一個員工，輸出 [Name] ID: [ID], Total Staff: [Total]。

範例輸入：

3

Alice

Bob

Charlie

範例輸出：

Alice ID: 1, Total Staff: 1

Bob ID: 2, Total Staff: 2

Charlie ID: 3, Total Staff: 3

題目：詳見題目說明。

主張(Claim)：先定義Employee類別及2個類別屬性，建構子用來初始化員工姓名及ID。

驗證(Re

Problem: **A. Employee ID System - AAI_PYTHON-E28_Employee_ID_System** | Submitted: **13:58** | Language: **Python 3** | Compilation: **successful**

Run result: **CORRECT**

證據(Evidence)：
蒐集主張中的相關證據

`class Employee`	定義員工類別，是物件導向程式設計的基礎結構 [2]。[2]
類別屬性（`total_employees`, `next_id`）	用於儲存全類別共享的資料，在此用於追蹤總數和下一個 ID [1]。[1]
`__init__(self, ...)`	類別的建構子，用於在物件建立時初始化實體屬性（如 `self.name`, `self.id`）[3]。[3]
實體屬性（`self.name`, `self.id`）	每個員工物件獨有的資料 [1]。[3]
類別名稱存取（`Employee.next_id`）	在實例方法中，使用類別名稱來存取和修改類別屬性 [1]。[1]
`input()` / `sys.stdin.readline()`	用於從使用者（或標準輸入）讀取資料 [6]。[6]
`int()`	用於將輸入的字串轉換為整數型別，以確定需要處理的員工數量 [7]，[6]。[7]
`for` 迴圈	用來重複執行建立新員工物件的過程 N 次 [8]。[8]
`print()` / f-string	用於按照指定的格式輸出結果，將變數值嵌入到字串中 [11]，[10]。[11]

推理(Reasoning)：
將證據連結主張的論述

1. 類別與屬性的建立

類別定義（`class Employee`）

- **用途：** 類別作為藍圖，定義了所有員工物件應具備的屬性（資料）與行為（方法）[1]。
- **實踐：** 我們使用 `class Employee:` 語法來建立這個藍圖 [2]。

類別屬性 (Class Attributes)

- **用途：** 類別屬性儲存的是所有物件共享的資料，非常適合用於處理「類別層級的資料」[3]。在本情境中，「下一個可用 ID」和「總員工數」正是全公司共享的資訊。
- **實踐：** 類別內初始化 `total_employees = 0` 和 `next_id = 1`。這些屬性將透過類別名稱（如 `Employee.next_id`）來存取與修改 [3]。

實體屬性 (Instance Attributes)

- **用途：** 實體屬性儲存的是每個獨立員工物件特有的資料，例如姓名和分配到的 ID [3]。
- **實踐：** 在建構子（`__init__`）內，我們透過 `self` 參數來操作這些屬性，設定 `self.name` 和 `self.id` [4]。

建構子 (`__init__(self, name)`)

- **用途：** 建構子是類別中的一個特殊實例函數，會在建立新的員工物件（即當我們執行 `Employee(姓名)` 時）時自動執行，負責初始化該物件的狀態 [2] [2]。
- **實踐：**
  1. 接收傳入的 `name` 參數，設定給實例屬性 `self.name`。
  2. 將當前類別屬性 `Employee.next_id` 的值，賦予給當前物件的實例屬性 `self.id`，完成 ID 分配。
  3. 透過類別名稱，更新共享的狀態：`Employee.next_id += 1` 和 `Employee.total_employees += 1` [3]。

`self` 參數

- **用途：** `self` 是所有實例函數（包括 `__init__`）的第一個參數，它代表正在被操作的物件本身 [4] [4]。
- **實踐：** 藉由 `self`，我們可以將傳入的姓名賦予給當前員工物件（`self.name`），並將分配的 ID 儲存到該員工物件中（`self.id`）。

複合設值運算 (`+=`)

- **用途：** 複合設值敘述可以方便地更新變數的值，例如 `a += 1` 等同於 `a = a + 1` [6]。
- **實踐：** 在 `__init__` 中，用於簡潔地實現計數器遞增邏輯，例如 `Employee.total_employees += 1`，這會直接修改類別層級的共享數據。

學習筆記(Learning Diary)：
請記下這個學習過程所學到的重要事項

1.定義類別 (`class Employee`)
2.物件的建立
3.實踐共享狀態